BitFlip

CloudSherpa

Software Architecture Specifications

Contents

1 Introduction

The CloudSherpa Software Architecture Specification elaborates on the architectural and other technical decisions made throughout the development process. Its purpose is to serve as a reference for CloudSherpa's architectural direction. The document is intended for both developers and stakeholders and serves as a mechanism for maintaining alignment between them.

2 Architectural Requirements

Terminology Note

In the source-code repository, the **Application Service** is implemented by the application named *service*. To distinguish this specific application from the general use of the term service within a service-oriented architecture, it is referred to throughout this document as the Application Service.

2.1 Architectural Patterns

2.1.1 Service Oriented Architecture

CloudSherpa follows the Service Oriented Architecture (SOA) pattern to promote decoupling between the subsystems responsible for cloud data ingestion, the subsystems responsible for processing and facilitating the vizualisation of this data and the subsystems used to perform intelligence operations. The CloudSherpa architecture identifies three distinct services:

- **Ingestion Service**
  The ingestion service is responsible for connecting to the respective cloud provider interfaces for obtaining data from deployments.
- **Application Service**
  The Service Service is responsible for the business logic of CloudSherpa, exe-

cuting tasks like data aggregation, authentication and communicating intelligence insights to the Frontend Service.

- **AI/ML Service**
  The AI/ML Service is responsible for tasks related to CloudSherpa intelligence, such as Cost and Usage forecasting as well as optimization recommendations.

As part of our architecture we have a Dashboard presentation component that consumes the result of CloudSherpa processing by ways of the Application Service.

2.2 Architectural Diagram

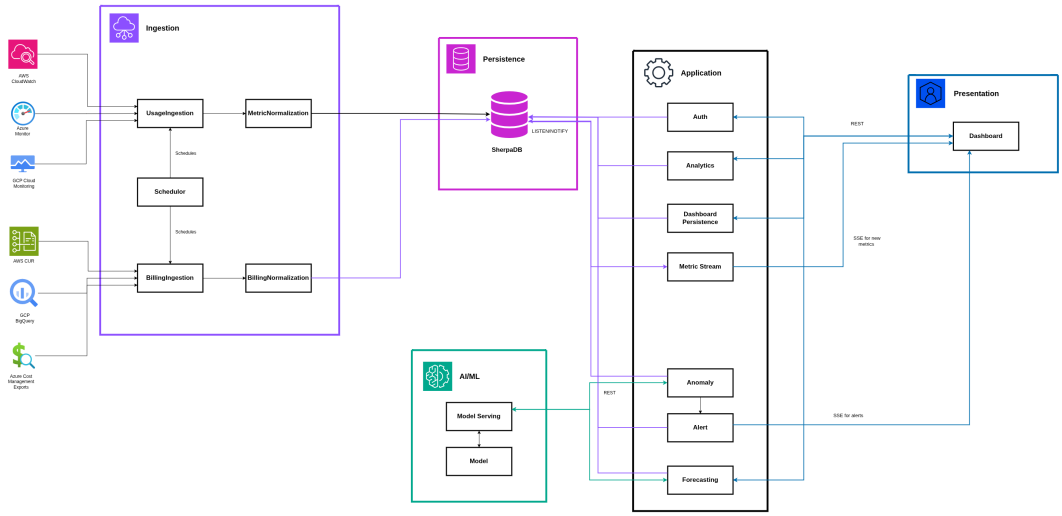

Figure 1: Architectural Diagram

2.3 Entity Relational Diagram

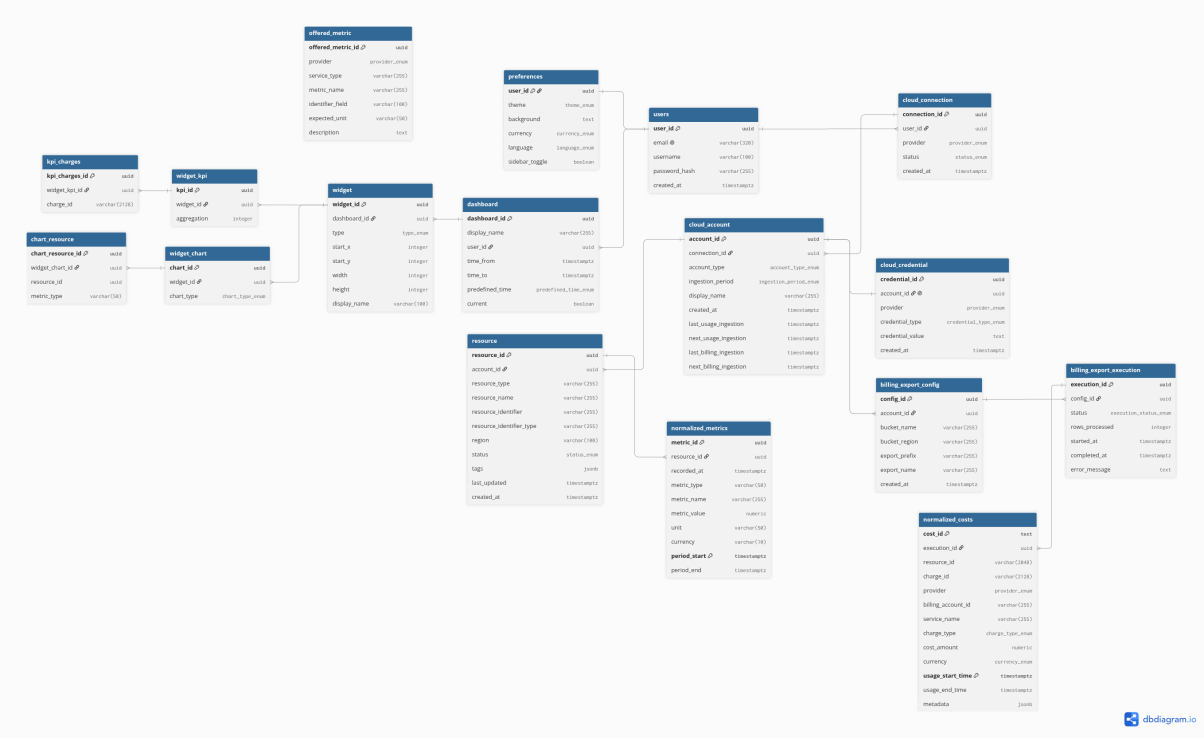

Figure 2: SherpaDB ERD

2.4 Design Patterns

2.4.1 Pipeline Design Pattern for AWS CUR Billing Ingestion

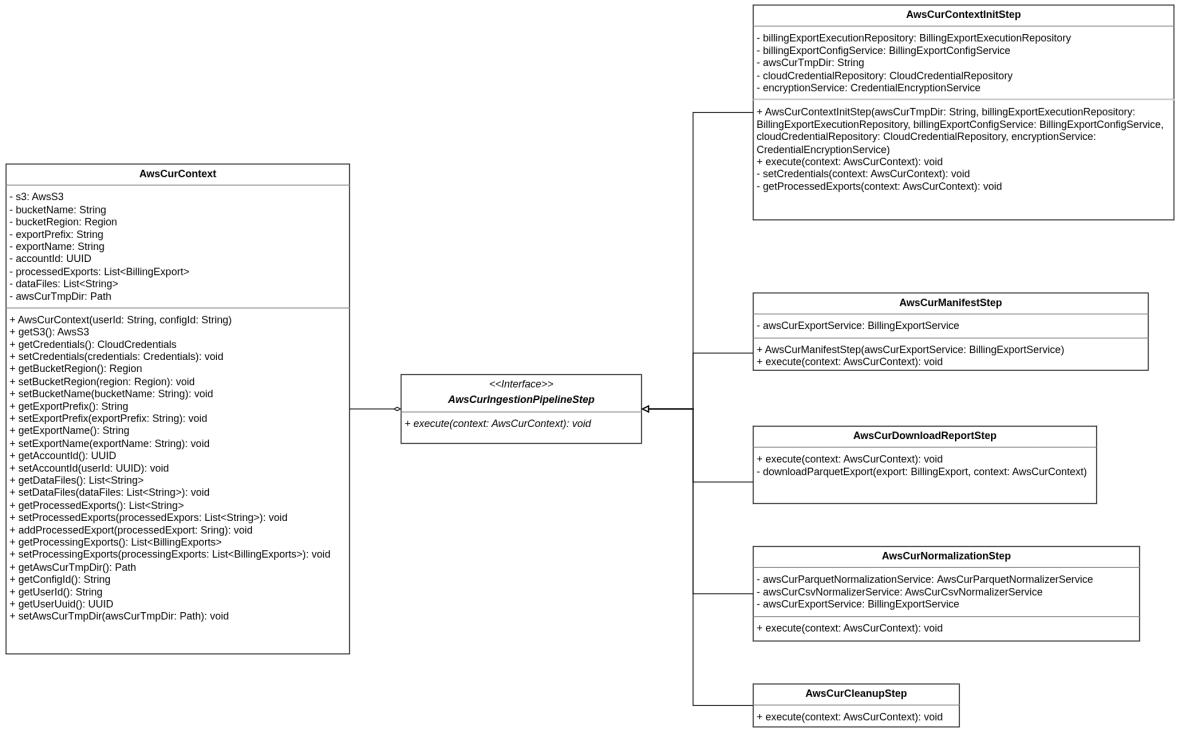

Figure 3: Pipeline Design Pattern For Billing Ingestion

Ingesting reports from a third party, such as a cloud provider, can introduce maintainability challenges. If the provider changes the API used to retrieve a report or modifies the report structure, the ingestion logic must be updated accordingly. Because the nature of future changes cannot be predicted, the ingestion process was implemented using the pipeline design pattern, which divides the process into a sequence of decoupled steps.

The pattern is reflected by the `AwsCurContext` object, which is passed through the pipeline and modified by each implementation of the `AwsCurIngestionPipelineStep` interface. Each step is responsible for a distinct part of the ingestion process.

This design improves maintainability by allowing individual steps to be modified, replaced, or extended without requiring substantial changes to the rest of the ingestion process. The modular structure also makes the billing ingestion process easier to test

and debug.

Note: Not all callers, associations, and aggregations are shown in the diagram.

2.4.2 Builder Design Pattern for Normalized Metrics

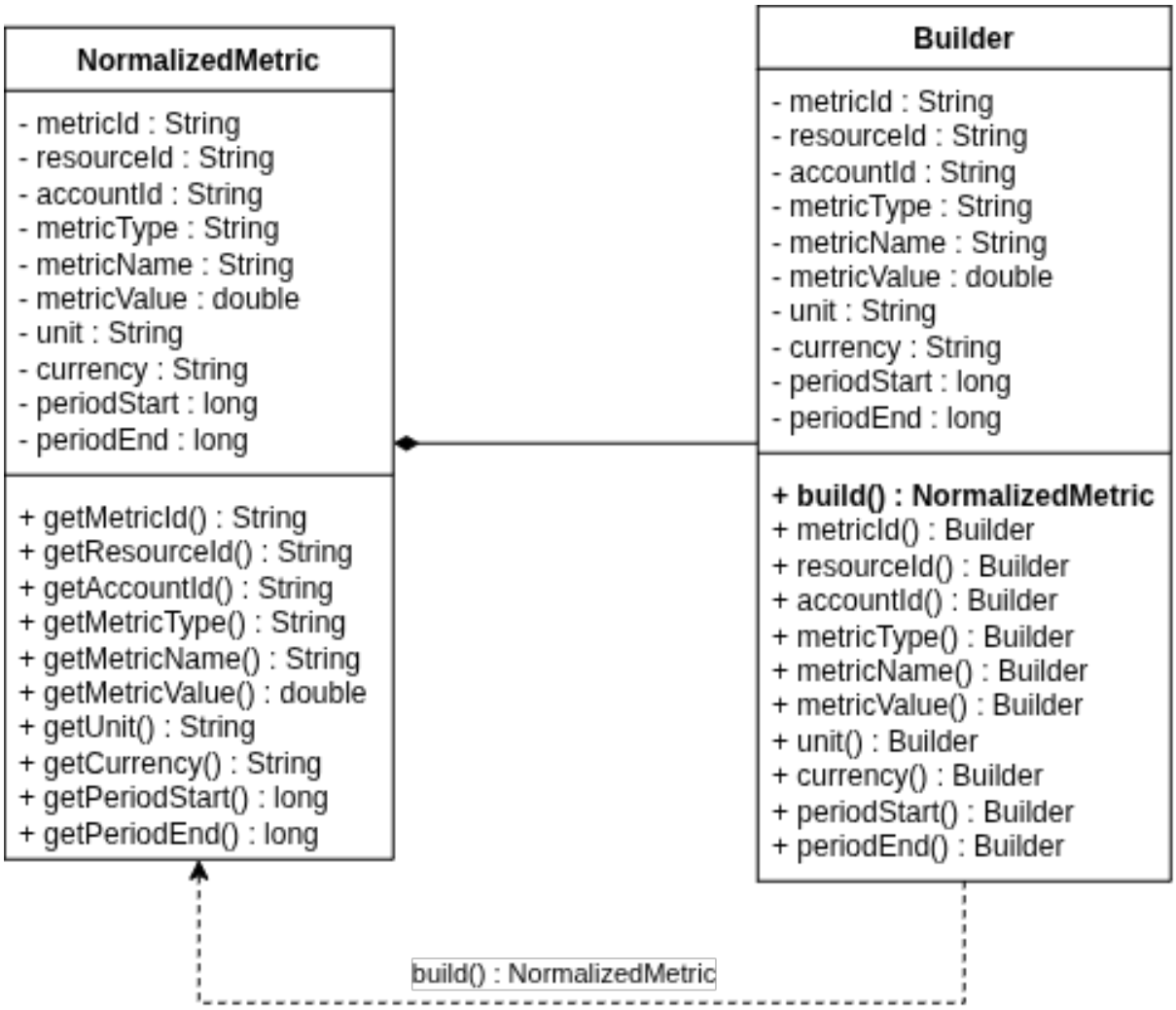

Figure 4: Builder Design Pattern for Normalized Metrics

The `NormalizedMetric` class uses a static nested Builder class to create objects using an interface. Each builder method sets a value and returns the builder, allowing methods to be chained together.

The `build()` method creates the final `NormalizedMetric` object using a private constructor. This makes the code easier to read and avoids the need for multiple constructors with different parameter combinations.

2.5 Constraints

2.5.1 Supported Cloud Providers

The system shall support AWS, Azure and GCP as cloud providers. Cloud providers not listed are outside of the scope of CloudSherpa.

2.5.2 Secure handling of cloud credentials

The system must follow the principle of least privelage when monitoring deployments accross cloud providers.

2.5.3 Single Role System (No RBAC)

CloudSherpa does not make provision for role-based users. The developer focus is to be able to deliver a refined system that achieves its primary goal well, which is to serve as a FinOps platform to improve the visibility of and provide insights to cloud spend.

2.5.4 Use Provider APIs Instead of Pushing Data from User Infrastructure

To minimize the operational complexity and security risks associated with collecting usage metrics from CloudSherpa users, cloud provider APIs are used to retrieve data through dedicated IAM or system users, rather than relying on software installed on users' compute instances to push the data.

2.5.5 Cost & Usage Latency

Due to the architectural decision to use cloud provider APIs rather than pushing data from user infrastructure, the frequency and recency of available usage information are subject to provider-specific delays.

A similar constraint applies to billing ingestion. However, unlike usage metrics, billing data cannot be obtained by pushing information directly from user infrastructure.

2.6 Mapping Quality Requirements to Architectural Decisions

Table 1: Mapping Quality Requirements to Architectural Decisions

NFR1: Performance	
NFR1.1: The system should load the main dashboard within 2 seconds and update visualization widgets within 1 second of a user modifying a time window, while seamlessly handling the throughput required to ingest metrics from 1,000 live deployments.	The ingestion service, which is responsible for processing incoming cloud metrics, is deployed independently from the service layer that handles dashboard requests for historical data and propagates live updates to the user interface. This separation prevents ingestion workloads from directly competing with dashboard requests for the same application resources, provided that the deployment allocates sufficient resources to each service. To support the requirement that the main dashboard load within two seconds and remain responsive, the system preprocesses and aggregates data before sending it to the dashboard, thereby reducing the number of data points that must be transferred and rendered. Once the dashboard receives the data, it is stored centrally and reused across multiple widgets rather than being duplicated, which reduces memory usage and avoids unnecessary processing.

NFR1.2: The ingestion pipeline shall process, normalize, and aggregate 100,000 raw billing and usage records from AWS, Azure, and GCP APIs into the unified internal schema within 60 seconds of data retrieval.	The architecture allows the ingestion service, which is responsible for processing and normalizing cloud billing and usage records, to be scaled independently according to ingestion volume and processing demand, hence improving performance by allocating sufficient resources. The scheduling subsystem allocates background jobs via JobRunr, which manages background asynchronous jobs, allowing multiple usage and ingestion jobs to be processed asynchronously. Our tenant per schema database allows those records to also be written to the database without conflict, due to the fact that separate tables are used for each user's usage and billing metrics. Therefore writes to one users' metric tables do not conflict with writes to other user's metric ingestion.
NFR2: Scalability	

NFR2.1: The system should successfully support 500 concurrent dashboard users and manage 1,000 active cloud connections without experiencing more than a 10% degradation in dashboard response times.	Data ingestion, processing, and normalization are performed by the independently deployable ingestion service, while dashboard requests are handled by the service layer. The service layer retrieves and aggregates historical data into a form optimized for dashboard rendering, reducing the amount of data transferred and processed for each user request. Because the ingestion and user-facing services are decoupled and operate in isolated execution environments, increases in the number of connected cloud accounts primarily affect the ingestion service rather than the dashboard request path. Each service can therefore be scaled independently according to its workload, helping the system support additional cloud connections and concurrent dashboard users without a significant degradation in response time.

NFR2.2: The time-series database architecture shall scale dynamically to accommodate a 200% year-over-year growth in stored historical cloud data without requiring manual partitioning or major architectural changes.	The system employs a schema-per-tenant database architecture to isolate each customer's data, improving scalability by reducing cross-tenant contention and allowing each tenant's workload to be managed independently. Within each tenant schema, historical cloud usage data is stored in TimescaleDB hypertables, which automatically partition data into time-based chunks as it is ingested. This automatic chunking eliminates the need for manual table partitioning and maintains efficient query performance as historical datasets grow. To further support long-term scalability, historical chunks are automatically compressed using TimescaleDB's native compression capabilities, significantly reducing storage requirements while preserving efficient queries. Together, these architectural decisions enable the time-series database to accommodate the projected growth in stored historical cloud data without requiring manual partitioning or major architectural changes.
NFR3: Security	

NFR3.1: All sensitive user data and stored cloud provider credentials must be encrypted using TLS 1.3 in transit, and the system must enforce Multi-Factor Authentication (MFA) on 100% of accounts.	All external requests pass through a single controlled entry point before reaching the system's internal services. TLS 1.3 is enforced at this boundary, ensuring that communication between the dashboard and backend services is encrypted in transit. Authentication is centralised through a shared identity and access management component, which requires multi-factor authentication for all user accounts and prevents unauthenticated requests from bypassing the authorisation process. Cloud credentials are stored only in encrypted format by the database using AES symmetric key cryptography.
NFR3.2: The system shall utilize secure, least-privilege identity access management (IAM) strategies, ensuring that CloudSherpa holds read-only access to client AWS, Azure, and GCP billing APIs and ensuring that none of CloudSherpa's requests require write permissions.	Third-party integrations are confined to a single service, allowing them to be implemented through consistent interfaces, governed by provider-specific security configurations, and monitored and audited from a central point. Permissions requests for IAM users include only necessary read-only permissions required for CloudSherpa to successfully discover and monitor selected services as authorized by the user.
NFR3.3: User metrics must remain separated such that no user gains access to resource details or cost and metric information from accounts not belonging to that user, as this information is potentially highly sensitive to individuals or corporations.	The architecture of our database, including our schema-per-tenant tables as well as strict session management and API authorization measures ensure that no user is capable of accessing any tenant specific information, including cost and resource metrics, not belonging to their accounts.
NFR4: Reliability	

NFR4.1: The system should achieve 99.9% uptime for both the dashboard and the metrics ingestion pipeline.	Reliability was considered in the architecture by decoupling the ingestion and application services, allowing failures and maintenance in one service to have a reduced impact on the other. However, further architectural improvements are required to meet the specified availability target. The circuit breaker pattern should be introduced for requests to third-party cloud provider APIs. When failures exceed a defined threshold, requests can be temporarily suspended to prevent repeated unsuccessful calls, resource exhaustion, and cascading failures. Database reliability can be improved through replication, automated failover, and appropriate data-partitioning mechanisms. The ingestion service can also be deployed across multiple instances behind a load balancer and configured for automatic scaling, particularly because it performs computationally intensive data-processing operations.
NFR4.2: In the event of a third-party cloud provider API failure or timeout (e.g., an AWS CloudWatch outage), the ingestion pipeline shall automatically retry the connection and keep track of the failure period, ensuring 0% duplication or metric loss once the provider is back online.	The CloudSherpa ingestion scheduler uses a recurring database poll to find accounts due for ingestion, using database fields to keep track of the last successful ingestion as well as the next scheduled events. JobRunr background jobs are configured to retry requests three times upon failure, and database success fields are only updated upon success, ensuring that upon subsequent scans outstanding jobs will be handled when the external provider is back online.
NFR5: Maintainability	

NFR5.1: The codebase must maintain a minimum of 80% automated test coverage across all core components, and the continuous integration pipeline should enable new feature deployments or bug fixes to reach production within 2 hours.	The modularity of our architecture improves the ease and pace at which tests can be developed, as large parts of our system remain loosely coupled. Separate CI workflows ensure that tests can be run for only affected subsystems quickly, aiding in verification of behaviour that allows us to quickly and confidently deploy bug fixes.
NFR5.2: The data normalization module must be sufficiently decoupled so that integrating a new cloud provider API, or updating an existing provider's changed billing schema, requires no more than 40 hours of developer effort.	The ingestion service, which is responsible for data ingestion and normalization, is decoupled from the rest of the system. Changes to this service do not require corresponding changes to other services, because communication between the ingestion and service layers occurs through a shared database schema that provides a standardized data contract.
NFR6: Usability	
NFR6.1: A new non-technical user should be able to complete core tasks, such as viewing forecasts or checking anomaly alerts, within 5 minutes of first using the system, achieving at least 85% user satisfaction during usability testing.	As the presentation service is decoupled from how the data is obtained from the Cloud providers, developers can engineer an intuitive user experience removed from the provider specific complexities. Our setup system guides users through connecting their accounts to CloudSherpa step by step and automates the more technical aspects such as resource discovery, minimal permissions generation, and configuration.

NFR6.2: The CloudSherpa dashboard shall render correctly on modern web browsers to ensure plain-language summaries remain highly accessible.	CloudSherpa was built using modern standards-compliant web technologies using features that are supported by all modern web browsers. Responsive CSS and the use of high quality component libraries such as Apache ECharts and Shadcn ensure that components render correctly on differing screen sizes and resolutions, and lighthouse reports are routinely run to ensure performance and accessibility. Component level testing as well as Playwright end-to-end tests verify performance and consistency across browsers, and user acceptance testing is further used to verify these properties in staging before changes are pushed to production.
NFR7: Availability	
NFR7.1: The system should be available 24/7, excluding scheduled maintenance periods not exceeding 2 hours per month.	By decoupling the Cloud Data Ingestion Service from the Application Service, historical data remains available to users even when cloud provider APIs are unavailable or ingestion fails because of an internal error. However, further architectural improvements are required to meet this availability target. As discussed under NFR 4.1, these improvements include introducing replication, load balancing, and multiple service instances to reduce single points of failure.
NFR7.2: The web dashboard and historical cost databases shall remain available and accessible to users even if one or more underlying cloud provider APIs (AWS, Azure, or GCP) experience a temporary outage.	Ingested cloud data is persisted in a database managed as part of the CloudSherpa deployment. The dashboard therefore retrieves historical cost and usage data from the local persistence layer rather than directly from third-party cloud provider APIs. As a result, temporary degradation or unavailability of AWS, Azure, or GCP APIs does not prevent users from accessing previously ingested historical data.

3 Technology Requirements

Terminology Note

Please note that references such as ***FR X*** or ***NFR X*** refer to recuirements documented in the CloudSherpa Software Requirement Specification documentation.

3.1 Next.js

We chose to use the Next.js full-stack framework for the CloudSherpa user-facing **Dashboard**.

3.1.1 Supporting functional requirements

- **FR 5. Web-Based Dashboard**
  Next.js is a framework intended for web development and thus facilitates the development of our web-based Dashboard as elicitated by **FR 5** in our SRS document. Reactive dashboard components are enabled by Next.js since it is built on top of the React framework.
- **FR 6. User and Account Management** Next.js middleware can inspect incoming HTTP requests and redirect users to the login page when an authorization token is absent.
- **FR 7. Alerts and Notifications** Next.js integrates with shadcn/ui, which provides reusable and accessible interface components for displaying alerts and notifications to users.

3.1.2 Supporting non-functional requirements

- **NFR 1. Performance**
  Next.js Server Components can be leveraged for computationally intensive Dashboard tasks, limiting client-side computation and improving dashboard responsiveness.
- **NFR 3. Security**
  Next.js Server Components allows developers to omit potentially sensitive information from the bundle that gets sent to the client browser. Next.js middleware

allows code to be executed server-side before a request is completed, allowing developers to intercept and inspect incoming client requests and perform custom actions such as redirecting the user to the login page when the authorization token is omitted from the request.

- **NFR 6. Usability** Next.js integrates with shadcn/ui, our chosen component library, which provides reusable and accessible interface components that support the development of a consistent and user-friendly dashboard.

3.1.3 Ease of development

- **App Router**
  The Next.js App Router simplifies navigation development by using the application's directory structure to define routes. It also provides conventions, such as route groups and private folders, that allow developers to organize related files and routes without necessarily affecting the resulting URL structure.

3.2 Spring Boot

We selected Spring Boot with Java 21 for the CloudSherpa Ingestion and Application services because of its performance benefits compared with the considered alternatives, such as NestJS, as well as Java's strong support for data structures and algorithms and the SDKs provided by the cloud providers that CloudsSherpa supports.

3.2.1 Supporting functional requirements

- **FR 1. Cloud Data Integration**
  AWS, GCP, and Azure provide Java SDKs that CloudSherpa uses for resource discovery and the ingestion of usage metrics and billing data. The JobRunr Java library provides persistent, multithreaded background job processing, which we use to schedule and execute usage and billing ingestion tasks.
- **FR 2. Data storage and processing**
  Spring Boot provides object-relational mapping support through JPA and Hibernate, allowing database records to be represented and accessed as Java objects. Hibernate also supports schema-based multitenancy, enabling tenant data to be isolated in separate database schemas. It also integrates with PostgreSQL's `LISTEN/NOTIFY` functionality, enabling event-driven processing in response to database changes.
- **FR 6. User and Account Management**
  Utilizing Spring Security Filter Chain for managing authorization, users and user-

related context can be obtained for each request as part of the Authenticatoin Context facilitated by Spring Security and the JWT protocol.

3.2.2 Supporting non-functional requirements

- **NFR 1. Performance**
  The spring boot multi-threaded architecture allows for competitive performance.
- **NFR 2. Scalability** Spring Batch supports the efficient processing of large data volumes through chunk-based processing and parallel execution.
- **NFR 3. Security**
  Spring Security provides a pipe-and-filter architecture to ensure that unauthenticated or unauthorized requests never reach the controllers but gets filtered out at an earlier stage in the Spring Security Filter Chain.
- **NFR 4. Reliability**
  Spring Boot Actuator exposes health and additional monitoring endpoints out of the box, enabling health-checks and uptime monitoring. Java custom exception hierarchies allows developers handle exceptions gracefully.
- **NFR 5. Maintainability**
  OOP and Java packages enables modular development making code easier to test, maintain and extend. Supporting testing frameworks such as *JUnit*, *Mockito* and *AssertJ* enable unit and integration testing.

3.2.3 Ease of development

- **Familiarity**
  As team BitFlip is comprised of 5 university students with a lot of exposure to the Java programming language, we are apply to better apply learnt concepts during the development of the most technical parts of CloudSherpa.

3.3 Docker and Docker Compose

3.3.1 Supporting non-functional requirements

- **NFR 1. Performance**
  Docker shares the operating system kernel which results in better performance than using traditional VMs.
- **NFR 2. Scalability**
  Containers can easily be scaled up or down. Containers can be replicated to facilitate distributed runtime environments.

- **NFR 3. Security**
  Our application runs as a set of isolated containers. Isolated containers have the benefit of increased security: when one part of the system is compromised it does not automatically mean that the entire system is compromised. Taking this into account, we explicitly handle all sensitive operations within a part of the system that runs in an isolated container and is less likely to be vulnerable to malicious exploits.

3.3.2 Ease of development

- **Portability**
  Docker was chosen to improve application portability by reducing its dependence on the environment in which it is executed.
- **Reproducibility**
  Containers isolate the application and its dependencies from the host environment. As a result, container behaviour should remain consistent across different environments, enabling more predictable and reproducible application execution.

3.4 PostgreSQL & TimescaleDB

We use **PostgreSQL** as our relational database management system (RDBMS), together with the TimescaleDB extension for storing and querying time-series data. A key reason for selecting PostgreSQL is that TimescaleDB is built on top of it, allowing time-series hypertables and conventional relational tables to coexist and be queried within a single database.

3.4.1 Supporting functional requirements

- **FR 2. Data Storage and Processing**
  PostgreSQL schemas define the structure and organization of data stored at rest. The TimescaleDB extension is used to efficiently store and query time-series data, while PostgreSQL's `LISTEN/NOTIFY` functionality supports event-driven communication when new usage metric data becomes available. A schema-per-tenant architecture is used to logically isolate each tenant's data within the database.

3.4.2 Supporting non-functional requirements

- **NFR 1. Performance**
  TimescaleDB improves the storage and query performance of time-series data through time-based partitioning and optimizations designed for time-series workloads.

3.5 FastAPI

FastAPI was chosen to facilitate communication between the forecasting model and the application service. Its Python runtime enables direct integration with the forecasting model (Chronos-2) and its supporting machine-learning libraries, while its performance and ease of development support the efficient implementation of the required API.

3.5.1 Supporting Functional Requirements

- **FR 3. Cost Prediction**
  The FastAPI framework facilitates REST-based communication between the Application Service and the AI/ML Intelligence Service. The Application Service sends time-series data and a specified prediction horizon to the forecasting model exposed through FastAPI, which returns predictions for the requested number of future timestamps.

3.5.2 Supporting Non-Functional Requirements

- **NFR 1. Performance** According to the official FastAPI documentation, its performance is comparable to that of high-performance frameworks built using technologies such as Node.js and Go. As FastAPI is primarily responsible for exposing the forecasting model through a REST API, its performance helps minimize the communication overhead between the Application Service and the AI/ML Intelligence Service.

3.5.3 Ease of Development

Built on Pydantic, FastAPI automates development tasks such as validation, serialization and endpoint documentation.

4 Deployment

4.1 Deployment Requirements

- **Live, Accessible System:**
  The system is accessible at *https://cloudsherpa.gjjcs.org*

- **Environment Parity:**
  - **Development Environment**
    BitFlip members develop CloudSherpa on their local machines. To ensure reproducibility and consistency across the team, the development environment is containerized using Docker and Docker Compose. This setup closely mirrors the production deployment environment, particularly in how container images are built and services are orchestrated. Team members who do not have sufficient local computing resources to run the complete containerized environment can instead make use of the development servers provided by the frameworks that we use (refer to section 3).
  - **Staging Environment**
    The CloudSherpa staging environment is hosted on an AWS EC2 instance, with deployment automated through GitHub Actions. It serves as a Quality Assurance (QA) environment in which the team can test the platform under conditions that closely resemble the production environment. Nginx-enforced HTTP Basic Authentication restricts access to authorized team members and stakeholders, preventing unintended public access to the staging deployment. The automated deployment workflow is triggered by a `push` event to the `staging` branch.
  - **Production Environment**
    The CloudSherpa production environment is hosted on an AWS EC2 instance, with deployment automated through GitHub Actions. The production deployment workflow is triggered by a `push` event to the `main` branch.
- **Containerisation:**
  Docker was selected as the containerisation platform for CloudSherpa, enabling the system to be developed and deployed as a set of isolated, portable services. Docker Compose is used to define and manage the platform's multi-container architecture. For deployment, AWS Elastic Container Registry (ECR) serves as a private, managed container registry that integrates seamlessly with the staging and production environments hosted on AWS.
- **Secrets Management:**
  Local `.env` files are used to manage secrets and environment-specific configuration. To ensure consistency across environments and enable configuration validation, the project uses environment variable schemas. *Varlock* was selected to define and manage these schemas, together with a custom validation script that verifies that all required environment variables are present. The Varlock schema files can be safely committed to version control because they define only the expected configuration structure and do not contain any secret values.

- **Rollback Strategy:**
  CloudSherpa container images are built and deployed automatically using GitHub Actions. Each image is tagged with the commit hash associated with the deployment, allowing deployments to be pinned to specific versions of the codebase. A deployment script accepts a commit hash as a command-line argument, pulls the corresponding images from the container registry, and deploys them. If a rollback is required, the commit hash of a previously stable deployment is supplied to the script, restoring the system to that version.

4.2 Deployment Diagram

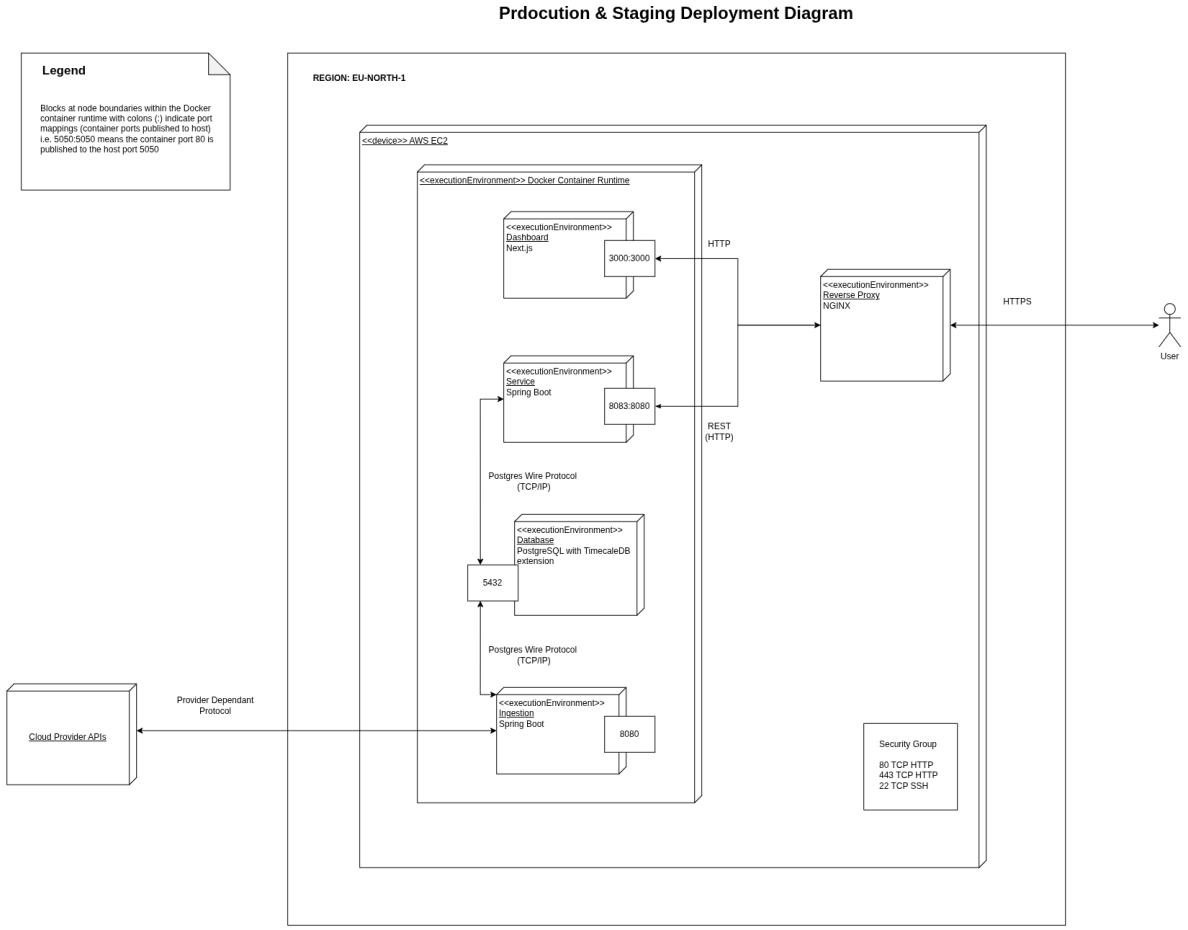

Figure 5: CloudSherpa Deployment Diagram (Production)

4.3 CI/CD Pipeline Diagram

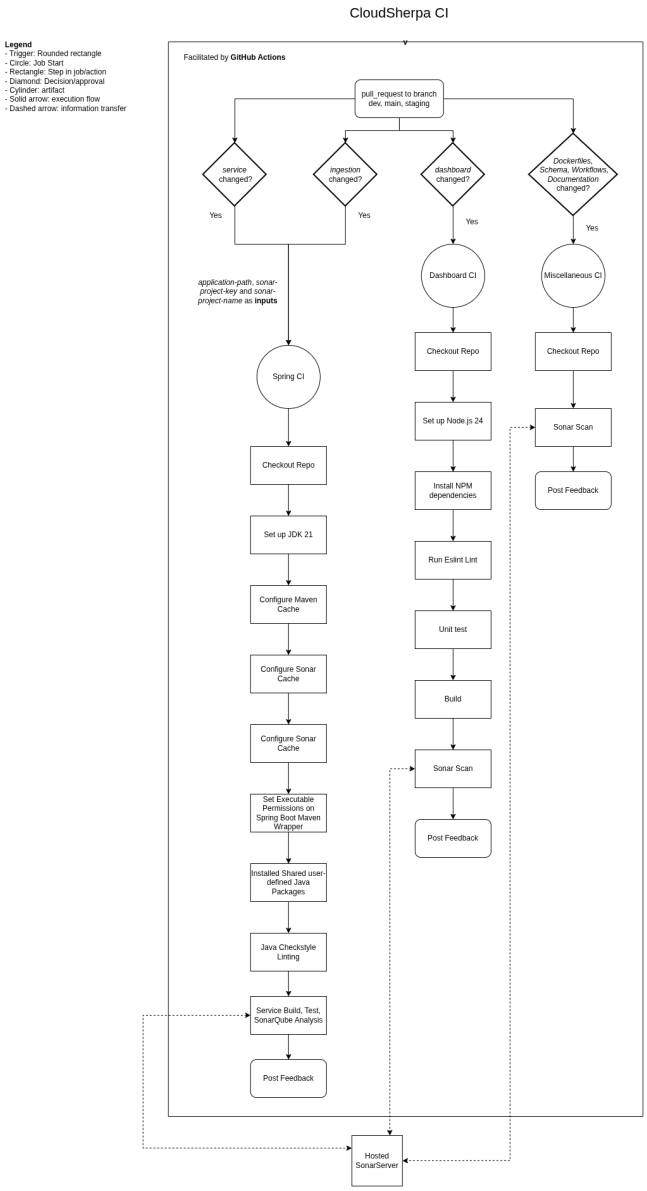

Figure 6: CI Pipeline Flow

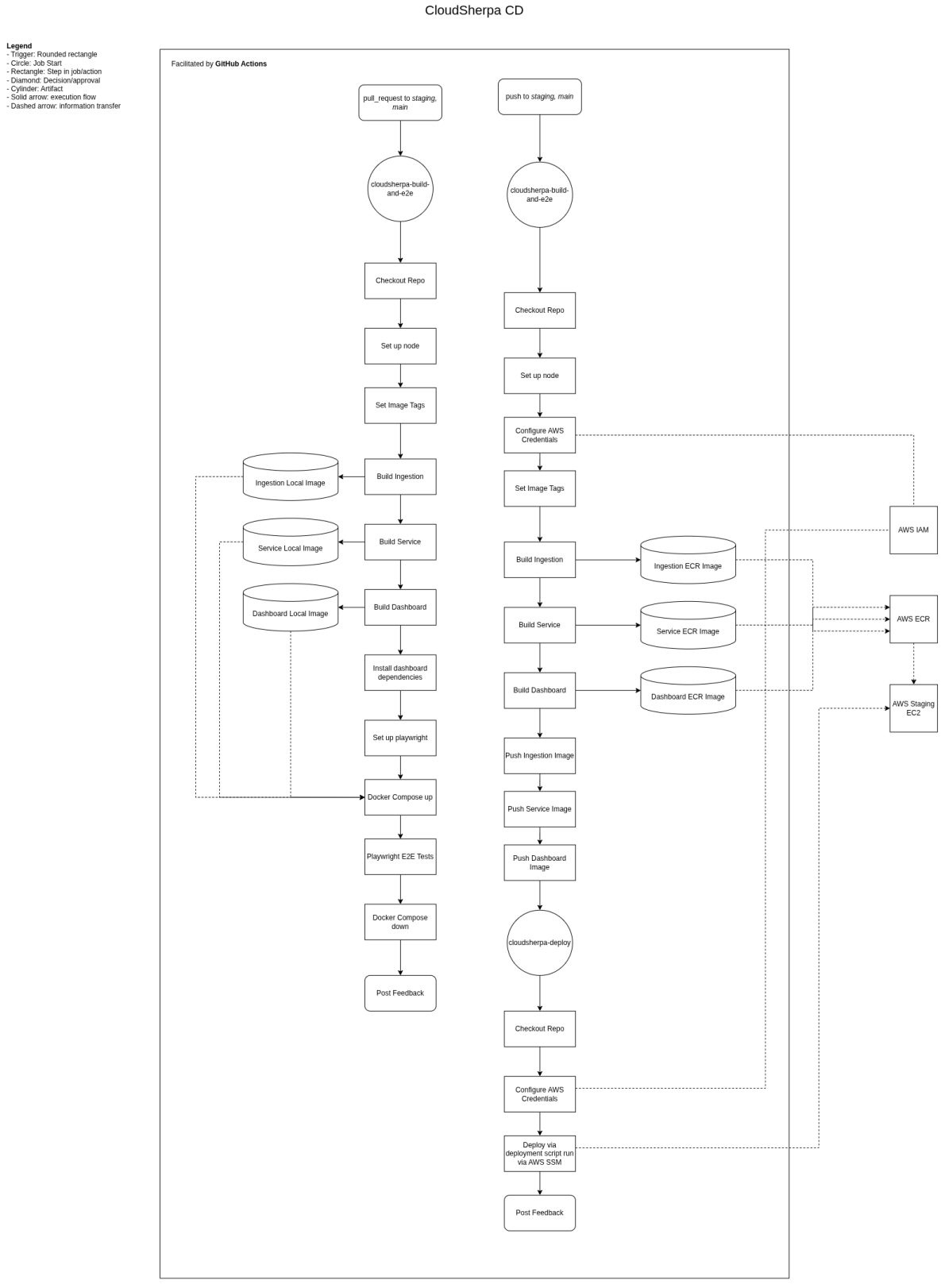

Figure 7: CD Pipeline Flow

A Appendix A: API Contracts

A.1 Ingestion Service

`/api/events/ingest`

Ingest real cloud usage and billing data

Request:

```
{
  "scopes": [
    {
      "provider": "string",
      "accountId": "string",
      "subscriptionId": "string",
      "projectId": "string",
      "billingAccountId": "string",
      "serviceScopes": [
        {
          "name": "string",
          "instances": [
            {
              "identifierName": "string",
              "values": [
                "string"
              ]
            }
          ],
          "metrics": [
            {
              "name": "string",
              "unit": "string"
            }
          ]
        }
      ]
    }
  ],
  "credentials": {
```

```
    "accessKey": "string",
    "secretKey": "string",
    "awsRegion": "string",
    "tenantId": "string",
    "clientId": "string",
    "clientSecret": "string",
    "projectId": "string",
    "serviceAccountJson": "string"
  },
  "from": "2026-06-28T20:17:14.812Z",
  "to": "2026-06-28T20:17:14.812Z",
  "period": 0,
  "includeBilling": true,
  "includeUsage": true,
  "userId": "3fa85f64-5717-4562-b3fc-2c963f66afa6"
}
```

Response:

```
{
  "usage": [
    {
      "recordId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "provider": "string",
      "accountId": "string",
      "subscriptionId": "string",
      "projectId": "string",
      "resourceId": "string",
      "resourceType": "string",
      "serviceName": "string",
      "region": "string",
      "metricName": "string",
      "value": 0.1,
      "unit": "string",
      "timestamp": "2026-06-28T20:17:14.814Z",
      "periodStart": "2026-06-28T20:17:14.814Z",
      "periodEnd": "2026-06-28T20:17:14.814Z",
      "dimensions": {
        "additionalProp1": "string",
        "additionalProp2": "string",
        "additionalProp3": "string"
      },
      "tags": {
```

```
        "additionalProp1": "string",
        "additionalProp2": "string",
        "additionalProp3": "string"
      },
      "ingestionTimestamp": "2026-06-28T20:17:14.814Z",
      "ingestionId": "string",
      "source": "string"
    }
  ],
  "billing": [
    {
      "recordId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "provider": "string",
      "accountId": "string",
      "subscriptionId": "string",
      "projectId": "string",
      "billingAccountId": "string",
      "serviceName": "string",
      "resourceId": "string",
      "resourceType": "string",
      "region": "string",
      "cost": 0.1,
      "usageQuantity": 0.1,
      "unit": "string",
      "currency": "string",
      "pricingModel": "string",
      "usageStartTime": "2026-06-28T20:17:14.814Z",
      "usageEndTime": "2026-06-28T20:17:14.814Z",
      "billingPeriodStart": "2026-06-28T20:17:14.814Z",
      "billingPeriodEnd": "2026-06-28T20:17:14.814Z",
      "tags": {
        "additionalProp1": "string",
        "additionalProp2": "string",
        "additionalProp3": "string"
      },
      "ingestionTimestamp": "2026-06-28T20:17:14.814Z",
      "ingestionId": "string",
      "source": "string"
    }
  ]
}
```

/api/events/ingest/mock

Generates and ingests deterministic mock cloud usage datafor development and integration testing.

Request:

```
{
  "scopes": [
    {
      "provider": "string",
      "accountId": "string",
      "subscriptionId": "string",
      "projectId": "string",
      "billingAccountId": "string",
      "serviceScopes": [
        {
          "name": "string",
          "instances": [
            {
              "identifierName": "string",
              "values": [
                "string"
              ]
            }
          ],
          "metrics": [
            {
              "name": "string",
              "unit": "string"
            }
          ]
        }
      ]
    }
  ],
  "credentials": {
    "accessKey": "string",
    "secretKey": "string",
    "awsRegion": "string",
    "tenantId": "string",
    "clientId": "string",
    "clientSecret": "string",
    "projectId": "string",
    "serviceAccountJson": "string"
```

```
  },
  "from": "2026-06-28T20:20:54.560Z",
  "to": "2026-06-28T20:20:54.560Z",
  "period": 0,
  "includeBilling": true,
  "includeUsage": true,
  "userId": "3fa85f64-5717-4562-b3fc-2c963f66afa6"
}
```

Response:

```
{
  "usage": [
    {
      "recordId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "provider": "string",
      "accountId": "string",
      "subscriptionId": "string",
      "projectId": "string",
      "resourceId": "string",
      "resourceType": "string",
      "serviceName": "string",
      "region": "string",
      "metricName": "string",
      "value": 0.1,
      "unit": "string",
      "timestamp": "2026-06-28T20:20:54.565Z",
      "periodStart": "2026-06-28T20:20:54.565Z",
      "periodEnd": "2026-06-28T20:20:54.565Z",
      "dimensions": {
        "additionalProp1": "string",
        "additionalProp2": "string",
        "additionalProp3": "string"
      },
      "tags": {
        "additionalProp1": "string",
        "additionalProp2": "string",
        "additionalProp3": "string"
      },
      "ingestionTimestamp": "2026-06-28T20:20:54.565Z",
      "ingestionId": "string",
      "source": "string"
    }
```

```
  ],
  "billing": [
    {
      "recordId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "provider": "string",
      "accountId": "string",
      "subscriptionId": "string",
      "projectId": "string",
      "billingAccountId": "string",
      "serviceName": "string",
      "resourceId": "string",
      "resourceType": "string",
      "region": "string",
      "cost": 0.1,
      "usageQuantity": 0.1,
      "unit": "string",
      "currency": "string",
      "pricingModel": "string",
      "usageStartTime": "2026-06-28T20:20:54.565Z",
      "usageEndTime": "2026-06-28T20:20:54.565Z",
      "billingPeriodStart": "2026-06-28T20:20:54.565Z",
      "billingPeriodEnd": "2026-06-28T20:20:54.565Z",
      "tags": {
        "additionalProp1": "string",
        "additionalProp2": "string",
        "additionalProp3": "string"
      },
      "ingestionTimestamp": "2026-06-28T20:20:54.565Z",
      "ingestionId": "string",
      "source": "string"
    }
  ]
}
```


/api/events/ingest/mockNoise

Generates and ingests mock cloud usage data using Reinstein Uhlenbeck noise for analytics, anomaly detection, and testing.

Request:


```
{
  "scopes": [
```

```
    {
      "provider": "string",
      "accountId": "string",
      "subscriptionId": "string",
      "projectId": "string",
      "billingAccountId": "string",
      "serviceScopes": [
        {
          "name": "string",
          "instances": [
            {
              "identifierName": "string",
              "values": [
                "string"
              ]
            }
          ],
          "metrics": [
            {
              "name": "string",
              "unit": "string"
            }
          ]
        }
      ]
    }
  ],
  "credentials": {
    "accessKey": "string",
    "secretKey": "string",
    "awsRegion": "string",
    "tenantId": "string",
    "clientId": "string",
    "clientSecret": "string",
    "projectId": "string",
    "serviceAccountJson": "string"
  },
  "from": "2026-06-28T20:23:56.974Z",
  "to": "2026-06-28T20:23:56.974Z",
  "period": 0,
  "includeBilling": true,
  "includeUsage": true,
  "userId": "3fa85f64-5717-4562-b3fc-2c963f66afa6"
}
```

Response:


```
{
  "usage": [
    {
      "recordId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "provider": "string",
      "accountId": "string",
      "subscriptionId": "string",
      "projectId": "string",
      "resourceId": "string",
      "resourceType": "string",
      "serviceName": "string",
      "region": "string",
      "metricName": "string",
      "value": 0.1,
      "unit": "string",
      "timestamp": "2026-06-28T20:23:56.980Z",
      "periodStart": "2026-06-28T20:23:56.980Z",
      "periodEnd": "2026-06-28T20:23:56.980Z",
      "dimensions": {
        "additionalProp1": "string",
        "additionalProp2": "string",
        "additionalProp3": "string"
      },
      "tags": {
        "additionalProp1": "string",
        "additionalProp2": "string",
        "additionalProp3": "string"
      },
      "ingestionTimestamp": "2026-06-28T20:23:56.980Z",
      "ingestionId": "string",
      "source": "string"
    }
  ],
  "billing": [
    {
      "recordId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "provider": "string",
      "accountId": "string",
      "subscriptionId": "string",
      "projectId": "string",
      "billingAccountId": "string",
      "serviceName": "string",
```

```
      "resourceId": "string",
      "resourceType": "string",
      "region": "string",
      "cost": 0.1,
      "usageQuantity": 0.1,
      "unit": "string",
      "currency": "string",
      "pricingModel": "string",
      "usageStartTime": "2026-06-28T20:23:56.980Z",
      "usageEndTime": "2026-06-28T20:23:56.980Z",
      "billingPeriodStart": "2026-06-28T20:23:56.980Z",
      "billingPeriodEnd": "2026-06-28T20:23:56.980Z",
      "tags": {
        "additionalProp1": "string",
        "additionalProp2": "string",
        "additionalProp3": "string"
      },
      "ingestionTimestamp": "2026-06-28T20:23:56.980Z",
      "ingestionId": "string",
      "source": "string"
    }
  ]
}
```

/api/events/ingest/aws/billing/cur

When database is seeded appropriately, ingest a billing report for testing purposes

Request:

```

No parameters
```

Response:

```
{
  "bucketName": "string",
  "bucketRegion": {
    "globalRegion": true
  },
  "exportPrefix": "string",
  "exportName": "string",
  "accountId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
```

```
  "processedExports": [
    "string"
  ],
  "configId": "string",
  "userId": "string",
  "dataFiles": [
    "string"
  ],
  "awsCurTmpDir": {
    "absolute": true,
    "parent": {
      "absolute": true,
      "root": {
        "absolute": true,
        "fileName": {
          "absolute": true,
          "fileSystem": {
            "open": true,
            "readOnly": true,
            "separator": "string",
            "rootDirectories": "string",
            "fileStores": "string",
            "userPrincipalLookupService": "string"
          },
          "nameCount": 0
        },
        "fileSystem": {
          "open": true,
          "readOnly": true,
          "separator": "string",
          "rootDirectories": "string",
          "fileStores": "string",
          "userPrincipalLookupService": "string"
        },
        "nameCount": 0
      },
      "fileName": {
        "absolute": true,
        "fileSystem": {
          "open": true,
          "readOnly": true,
          "separator": "string",
          "rootDirectories": "string",
          "fileStores": "string",
```

```
          "userPrincipalLookupService": "string"
        },
        "nameCount": 0
      },
      "fileSystem": {
        "open": true,
        "readOnly": true,
        "separator": "string",
        "rootDirectories": "string",
        "fileStores": "string",
        "userPrincipalLookupService": "string"
      },
      "nameCount": 0
    },
    "root": {
      "absolute": true,
      "fileName": {
        "absolute": true,
        "fileSystem": {
          "open": true,
          "readOnly": true,
          "separator": "string",
          "rootDirectories": "string",
          "fileStores": "string",
          "userPrincipalLookupService": "string"
        },
        "nameCount": 0
      },
      "fileSystem": {
        "open": true,
        "readOnly": true,
        "separator": "string",
        "rootDirectories": "string",
        "fileStores": "string",
        "userPrincipalLookupService": "string"
      },
      "nameCount": 0
    },
    "fileName": {
      "absolute": true,
      "fileSystem": {
        "open": true,
        "readOnly": true,
        "separator": "string",
```

```
        "rootDirectories": "string",
        "fileStores": "string",
        "userPrincipalLookupService": "string"
      },
      "nameCount": 0
    },
    "fileSystem": {
      "open": true,
      "readOnly": true,
      "separator": "string",
      "rootDirectories": "string",
      "fileStores": "string",
      "userPrincipalLookupService": "string"
    },
    "nameCount": 0
  },
  "userUuid": "3fa85f64-5717-4562-b3fc-2c963f66afa6"
}
```

/api/cloud-resources/services

Retrieves a list of all services that can be monitored by CloudSherpa for a given provider

Request:

```
{
    "string"
}
```

Response:

```
{
    "string"
}
```

/api/cloud-resources/resources/provider

Retrieves all discoverable resources for a cloud provider using the supplied credentials

Request:

```
{
  "accessKey": "string",
  "secretKey": "string",
  "awsRegion": "string",
  "tenantId": "string",
  "clientId": "string",
  "clientSecret": "string",
  "projectId": "string",
  "serviceAccountJson": "string"
}
```

Response:

```
{
  "resourceId": "string",
  "name": "string",
  "resourceType": "string",
  "serviceCategory": "string",
  "tags": {
    "additionalProp1": "string",
    "additionalProp2": "string",
    "additionalProp3": "string"
  }
}
```

/api/cloud-resources/aws/permissions

Generates a least-privilege AWS IAM policy for the selected services

Request:

```
[
  "string"
]
```

Response:

```
{
    string
}
```

A.2 Service

/auth/register

Registers a new user.

Request:

```
{
  "email": "string",
  "username": "string",
  "password": "string"
}
```

Response:

```
200 User Successfully registered
400 Invalid input
409 Email already in use
```

/auth/logout

Logout User

Request:

```
No parameters
```

Response:

```
200 Successfully Logged Out
```

/auth/login

Login User

Request:

```
{
  "email": "string",
  "password": "string"
}
```

Response:

```
{
  "userId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "email": "string",
  "username": "string",
  "token": "string"
}
```

/auth/me

Get current authenticated user

Request:

```
No parameters
```

Response:

```
{
  "userId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "email": "string",
  "username": "string",
  "token": "string"
}
```

/analytics/resource-names

Get resources

Request:

```
No parameters
```

Response:

```
[
  {
    "resourceId": "string",
    "resourceName": "string"
  }
```

```
]
```

/analytics/historical

Get authorative metric data

Request:

```
from : string
to : string
interval : string
```

Response:

```
[
  {
    "resourceId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "metricType": "string",
    "metricName": "string",
    "metricValue": 0,
    "unit": "string",
    "periodStart": "2026-07-30T21:20:27.976Z",
    "periodEnd": "2026-07-30T21:20:27.976Z",
    "sampleCount": 0
  }
]
```

/dashboards/dashboardId/layout

Update the sizes and positions of all widgets on a dashboard

Request:

```
dashboardId : string

[
  {
    "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "x": 0,
    "y": 0,
    "w": 0,
    "h": 0
```

```
  }
]
```

Response:

```
200      OK
```

/dashboards

Get all user dashboards with their widgets

Request:

```
No parameters
```

Response:

```
[
  {
    "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "displayName": "string",
    "timeFrom": "2026-07-30T21:24:40.684Z",
    "timeTo": "2026-07-30T21:24:40.684Z",
    "predefinedTime": "T_5_MIN",
    "current": true,
    "widgets": [
      {
        "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
        "widgetType": "KPI",
        "displayName": "string",
        "startX": 0,
        "startY": 0,
        "width": 0,
        "height": 0,
        "chartType": "gauge_chart",
        "resourceId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
        "metricType": "string"
      },
      {
        "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
        "widgetType": "KPI",
        "displayName": "string",
```

```
        "startX": 0,
        "startY": 0,
        "width": 0,
        "height": 0,
        "chargeIds": [
          "string"
        ],
        "aggregationWindowDays": 0
      }
    ]
  }
]
```

/dashboards/dashboardId/widgets

Add a new widget to the dashboard

Request:

```
dashboardId : string

{
  "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "widgetType": "KPI",
  "displayName": "string",
  "startX": 0,
  "startY": 0,
  "width": 0,
  "height": 0,
  "chartType": "gauge_chart",
  "resourceId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "metricType": "string"
}
```

Response:

```
{
  "widgetType": "string"
}
```

/dashboards/widgets/widgetId/config

Update a widget's visual or data configuration

Request:

```
widgetId : string

{
  "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "widgetType": "KPI",
  "displayName": "string",
  "chartType": "gauge_chart",
  "resourceId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "metricType": "string"
}
```

Response:

```
200 OK
```

/dashboards/dashboardId

Delete a dashboard

Request:

```
dashboardtId : string
```

Response:

```
204 No Content
```

/dashboards/widgets/widgetId

Delete a widget from a dashboard

Request:

```
widgetId : string
```

Response:

```
204 No Content
```

/billing/kpis/preview

Returns a preview value for a KPI card using selected resources, date range, and aggregation

Request:

```
{
  "chargeIds": [
    "string"
  ],
  "from": "string",
  "to": "string",
  "aggregation": "string"
}
```

Response:

```
{
  "value": 0,
  "currency": "string",
  "selectedChargeCount": 0,
  "timeLabel": "string",
  "updatedAt": "string"
}
```

/billing/charges

Get billing charges

Request:

```
No parameters
```

Response:

```
[
```

```
  {
    "resourceId": "string",
    "chargeId": "string",
    "service": "string",
    "provider": "AWS"
  }
]
```

/api/cloud-resources/services

Retrieves a list of all supported services for a given cloud provider

Request:

```
{
  "string"
}
```

Response:

```
{
  "string"
}
```

/api/cloud-resources/resources/provider

Retrieves all discoverable resources for the specified cloud provider using the supplied credentials

Request:

```
{
  "services": [
    "string"
  ],
  "credentials": {
    "accessKey": "string",
    "secretKey": "string",
    "awsRegion": "string",
    "tenantId": "string",
    "clientId": "string",
    "clientSecret": "string"
```

```
  }
}
```

Response:

```
{
  "resourceId": "string",
  "name": "string",
  "resourceType": "string",
  "serviceCategory": "string",
  "region": "string",
  "tags": {
    "additionalProp1": "string",
    "additionalProp2": "string",
    "additionalProp3": "string"
  }
}
```

/api/cloud-resources/aws/permissions

Generates a least-privilege AWS IAM policy for the selected AWS services

Request:

```
[
  "string"
]
```

Response:

```
{
  "string"
}
```

/preferences/theme

Request:

```
No parameters
```

Response:

```
{
  "additionalProp1": "string",
  "additionalProp2": "string",
  "additionalProp3": "string"
}
```

/preferences/theme

Request:

```
No parameters
```

Response:

```
{
  "additionalProp1": "string",
  "additionalProp2": "string",
  "additionalProp3": "string"
}
```